
byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
223 ms
STDIN
Input for the program (Optional)
Output:
-- EXPERIMENT 6
-- AIM: Stored procedure transfer_employee with validation and
-- error handling; salary validation and audit logging triggers.
-- ==
-- Audit table
CREATE TABLE IF NOT EXISTS Salary_Audit (
audit_id INT AUTO_INCREMENT PRIMARY KEY,
emp_id INT NOT NULL,
old_salary DECIMAL(10,2),
new_salary DECIMAL(10,2),
changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
action VARCHAR(30)
);
-- Remove existing routines/triggers to permit re-running this section.
DROP PROCEDURE IF EXISTS transfer_employee;
DROP TRIGGER IF EXISTS validate_employee_salary;
DROP TRIGGER IF EXISTS validate_employee_salary_update;
DROP TRIGGER IF EXISTS salary_audit_trigger;
DELIMITER //
CREATE PROCEDURE transfer_employee(
IN p_emp_id INT,
IN p_new_dept_id INT
)
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
| 101 | Aarav | CSE |
+--------+----------+-----------+
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
| 101 | Aarav | CSE |
+--------+----------+-----------+
| Trigger | Event | Table | Statement |
| validate_employee_salary | INSERT | employee | BEGIN
IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000';
END IF;



Ctrl + K

PARUL NAMDEV

queries.sql

MySQL

Run

I/O

AI Agent

223 ms

```
CREATE PROCEDURE transfer_employee(
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_employee_count INT DEFAULT 0;
    DECLARE v_department_count INT DEFAULT 0;
    DECLARE v_current_dept INT DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_employee_count
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_employee_count = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Error: Employee does not exist.';
    END IF;

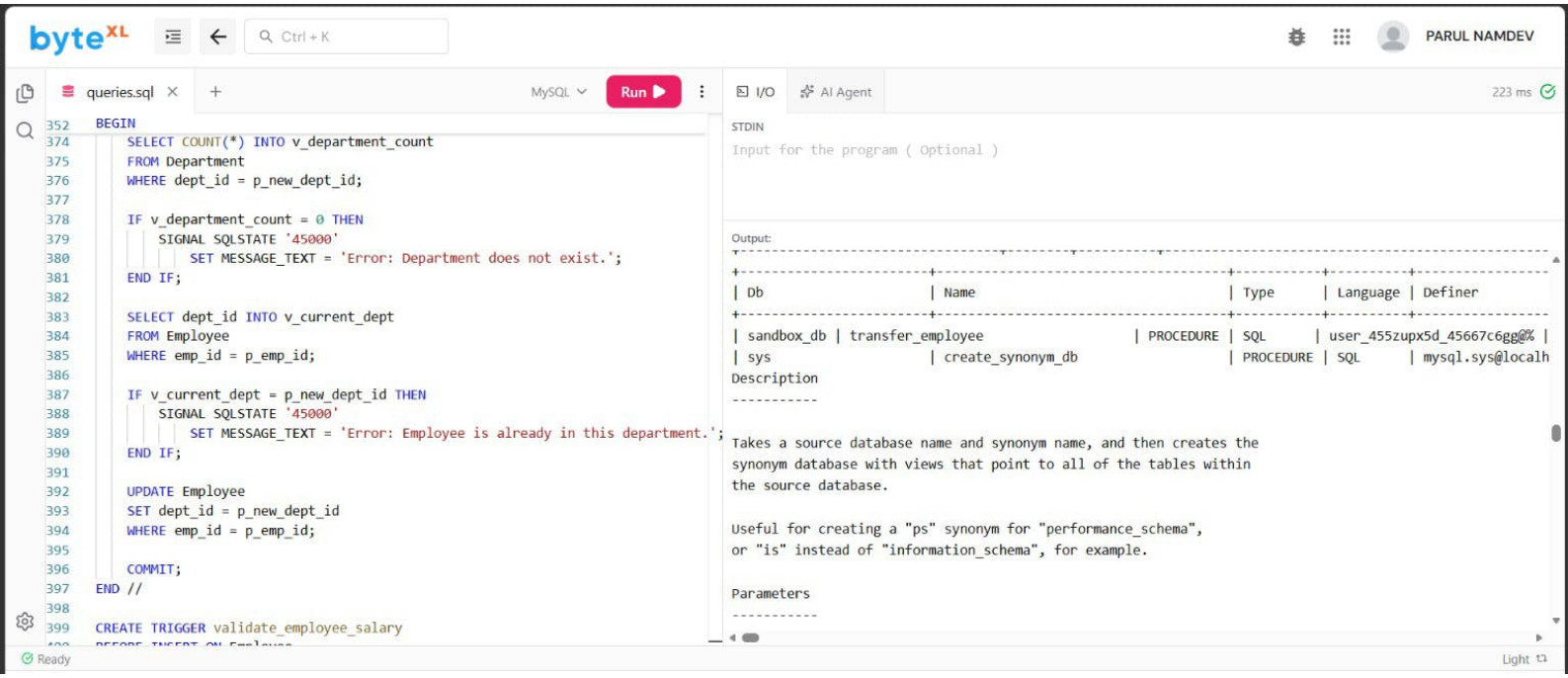
    SELECT COUNT(*) INTO v_department_count
    FROM Department
```

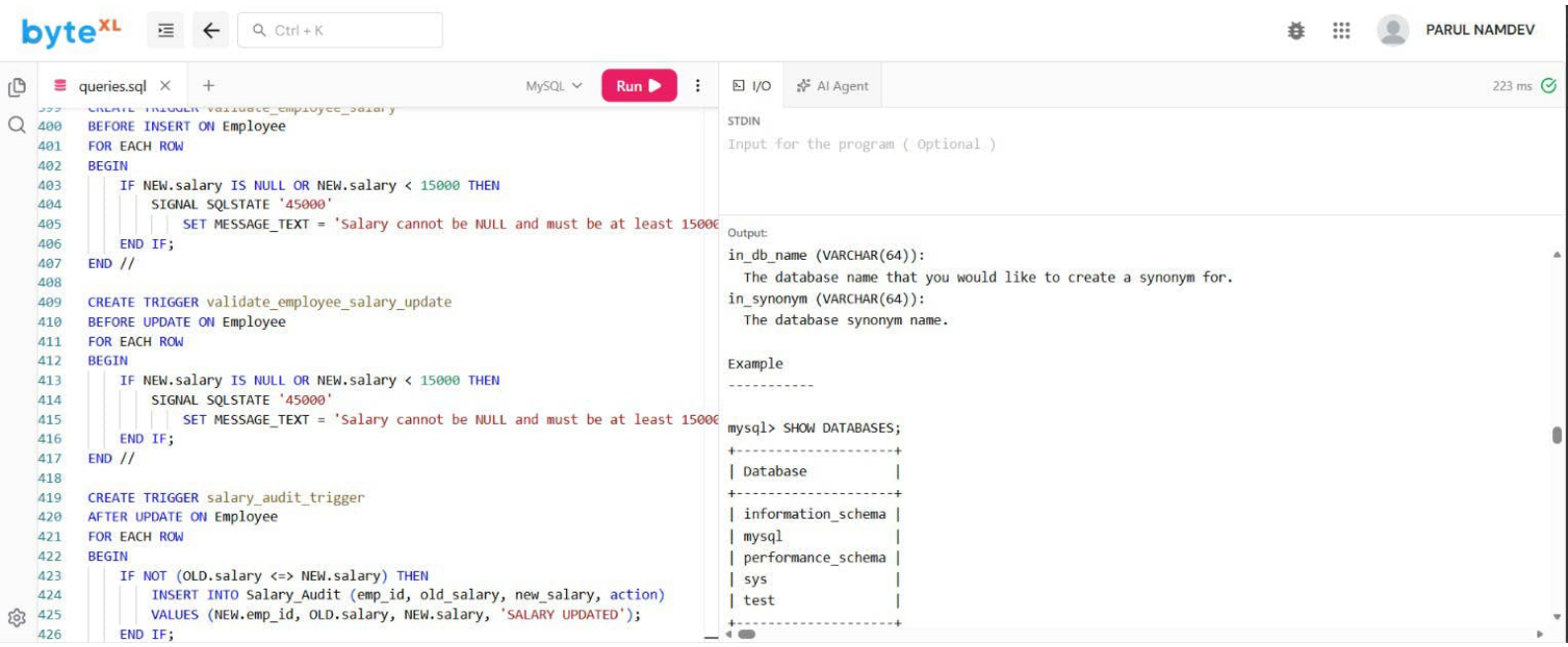
STDIN

Input for the program (Optional)

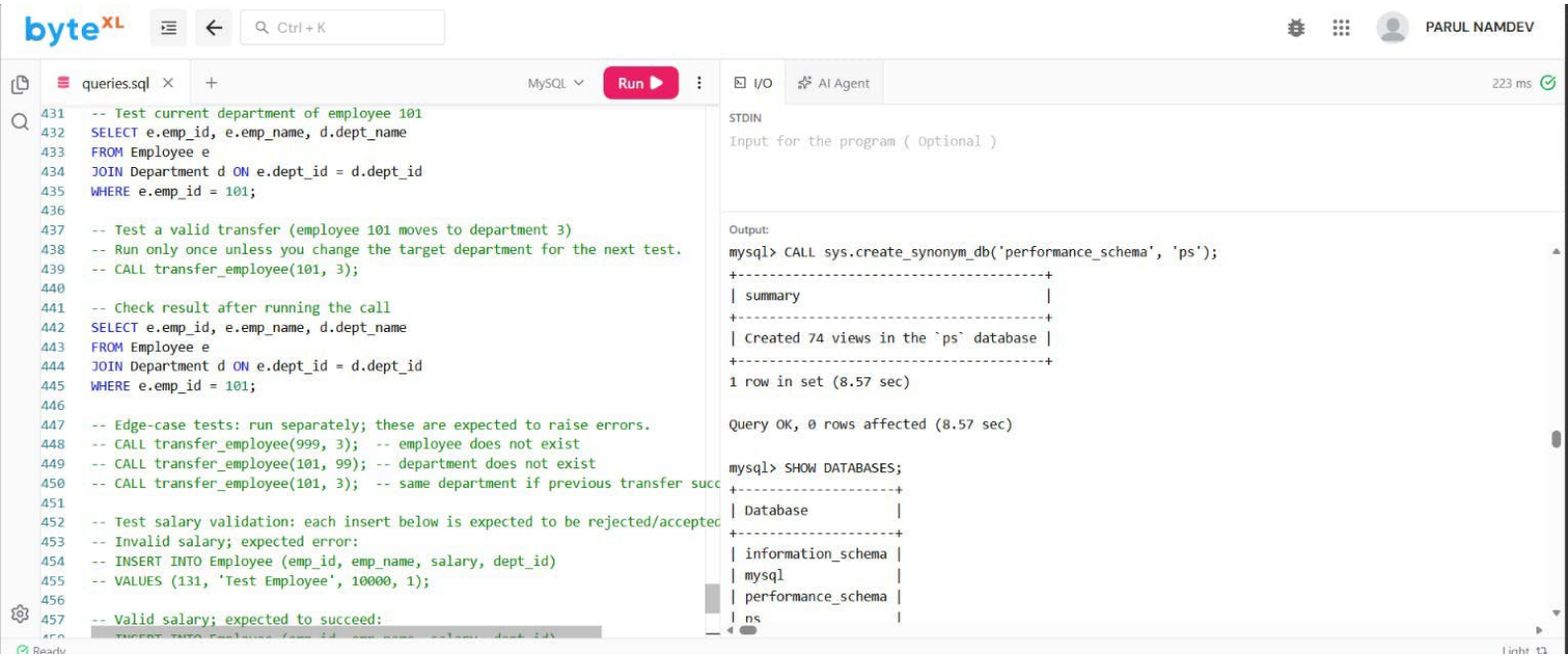
Output:

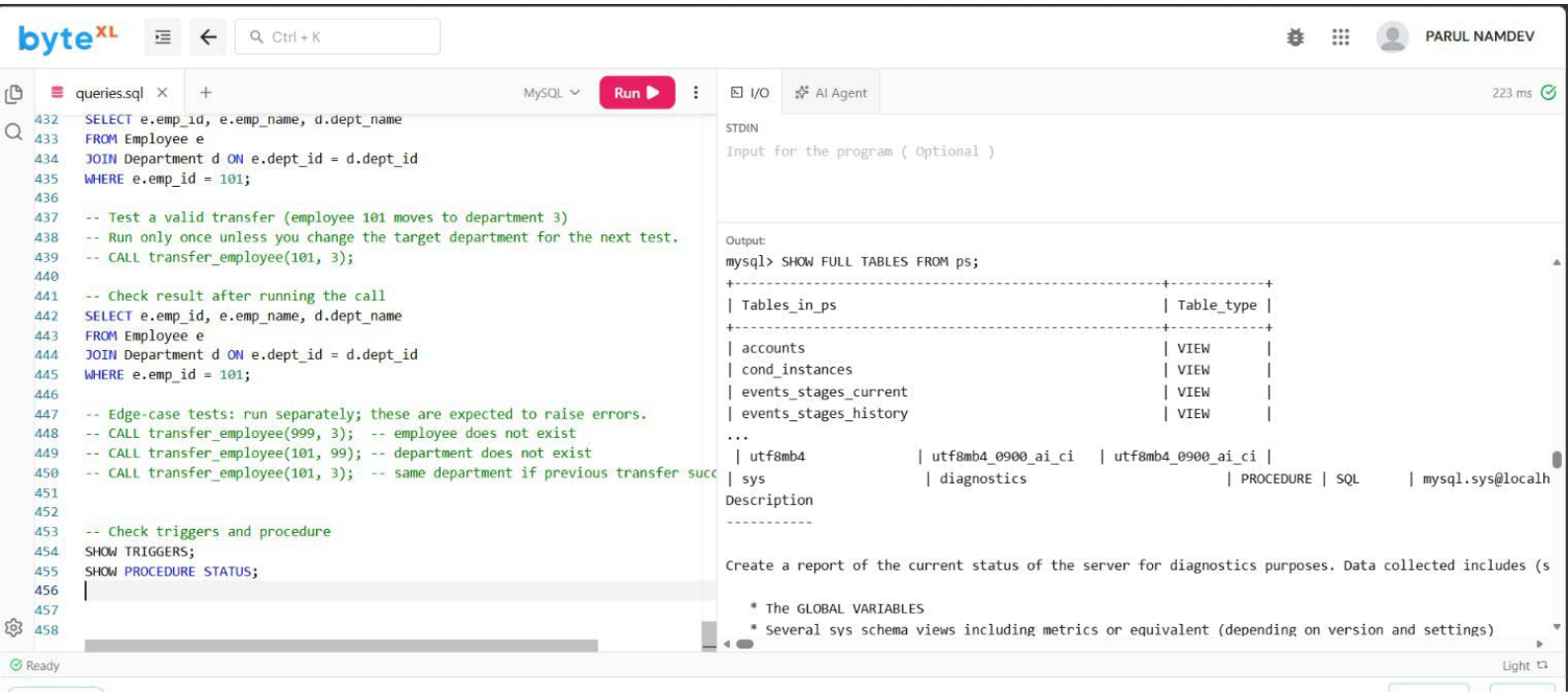
```
+---------------------------------+--------+----------+-----------------------------------------------------
| Trigger                         | Event  | Table    | Statement
+---------------------------------+--------+----------+-----------------------------------------------------
| validate_employee_salary        | INSERT | employee | BEGIN
    IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000';
    END IF;
END                     | BEFORE | 2026-10-14 14:49:28.56 | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_Z
| validate_employee_salary_update | UPDATE | employee | BEGIN
    IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000';
    END IF;
END                     | BEFORE | 2026-10-14 14:49:28.57 | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_Z
| salary_audit_trigger            | UPDATE | employee | BEGIN
    IF NOT (OLD.salary <=> NEW.salary) THEN
        INSERT INTO Salary_Audit (emp_id, old_salary, new_salary, action)
```


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
223 ms
BEGIN
SELECT COUNT(*) INTO v_department_count
FROM Department
WHERE dept_id = p_new_dept_id;
IF v_department_count = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Error: Department does not exist.';
END IF;
SELECT dept_id INTO v_current_dept
FROM Employee
WHERE emp_id = p_emp_id;
IF v_current_dept = p_new_dept_id THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Error: Employee is already in this department.';
END IF;
UPDATE Employee
SET dept_id = p_new_dept_id
WHERE emp_id = p_emp_id;
COMMIT;
END //
CREATE TRIGGER validate_employee_salary
STDIN
Input for the program (Optional)
Output:
| Db | Name | Type | Language | Definer
| sandbox_db | transfer_employee | PROCEDURE | SQL | user_455zupx5d_45667c6gg@% |
| sys | create_synonym_db | PROCEDURE | SQL | mysql.sys@localh
Description
Takes a source database name and synonym name, and then creates the
synonym database with views that point to all of the tables within
the source database.
Useful for creating a "ps" synonym for "performance_schema",
or "is" instead of "information_schema", for example.
Parameters
Ready
Light




```
400 BEFORE INSERT ON Employee
401 FOR EACH ROW
402 BEGIN
403     IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
404         SIGNAL SQLSTATE '45000'
405             SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000
406     END IF;
407 END //
408
409 CREATE TRIGGER validate_employee_salary_update
410 BEFORE UPDATE ON Employee
411 FOR EACH ROW
412 BEGIN
413     IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
414         SIGNAL SQLSTATE '45000'
415             SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000
416     END IF;
417 END //
418
419 CREATE TRIGGER salary_audit_trigger
420 AFTER UPDATE ON Employee
421 FOR EACH ROW
422 BEGIN
423     IF NOT (OLD.salary <=> NEW.salary) THEN
424         INSERT INTO Salary_Audit (emp_id, old_salary, new_salary, action)
425         VALUES (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATED');
426     END IF;
```


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
223 ms
STDIN
Input for the program (Optional)
Output:
-- Test current department of employee 101
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
-- Test a valid transfer (employee 101 moves to department 3)
-- Run only once unless you change the target department for the next test.
-- CALL transfer_employee(101, 3);
-- Check result after running the call
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
-- Edge-case tests: run separately; these are expected to raise errors.
-- CALL transfer_employee(999, 3); -- employee does not exist
-- CALL transfer_employee(101, 99); -- department does not exist
-- CALL transfer_employee(101, 3); -- same department if previous transfer succ
-- Test salary validation: each insert below is expected to be rejected/accepted
-- Invalid salary; expected error:
-- INSERT INTO Employee (emp_id, emp_name, salary, dept_id)
-- VALUES (131, 'Test Employee', 10000, 1);
-- Valid salary; expected to succeed:
mysql> CALL sys.create_synonym_db('performance_schema', 'ps');
| summary
| Created 74 views in the `ps` database |
1 row in set (8.57 sec)
Query OK, 0 rows affected (8.57 sec)
mysql> SHOW DATABASES;
| Database
| information_schema |
| mysql |
| performance_schema |
| ps |


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
223 ms
STDIN
Input for the program (Optional)
Output:
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
-- Test a valid transfer (employee 101 moves to department 3)
-- Run only once unless you change the target department for the next test.
-- CALL transfer_employee(101, 3);
-- Check result after running the call
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
-- Edge-case tests: run separately; these are expected to raise errors.
-- CALL transfer_employee(999, 3); -- employee does not exist
-- CALL transfer_employee(101, 99); -- department does not exist
-- CALL transfer_employee(101, 3); -- same department if previous transfer succ
-- Check triggers and procedure
SHOW TRIGGERS;
SHOW PROCEDURE STATUS;
mysql> SHOW FULL TABLES FROM ps;
Tables_in_ps	Table_type
accounts	VIEW
cond_instances	VIEW
events_stages_current	VIEW
events_stages_history	VIEW
...
| utf8mb4 | utf8mb4_0900_ai_ci | utf8mb4_0900_ai_ci |
| sys | diagnostics | PROCEDURE | SQL | mysql.sys@localh
Description
Create a report of the current status of the server for diagnostics purposes. Data collected includes (s
* The GLOBAL VARIABLES
* Several sys schema views including metrics or equivalent (depending on version and settings)
Ready
Light